

Implementing Lambdas with Substitution

CS 350

Dr. Joseph Eremondi

March 16, 2025

Broad Strategy

Broad Strategy

Implementation Strategy

- Have the the machinery we need to represent functions

Implementation Strategy

- Have the the machinery we need to represent functions
 - Treat functions as values

Implementation Strategy

- Have the the machinery we need to represent functions
 - Treat functions as values
 - Dynamic type checks

Implementation Strategy

- Have the the machinery we need to represent functions
 - Treat functions as values
 - Dynamic type checks
 - Substitution for replacing function arguments

Implementation Strategy

- Have the the machinery we need to represent functions
 - Treat functions as values
 - Dynamic type checks
 - Substitution for replacing function arguments
 - Already have from Let

Implementation Strategy

- Have the the machinery we need to represent functions
 - Treat functions as values
 - Dynamic type checks
 - Substitution for replacing function arguments
 - Already have from Let
- Need to decide:

Implementation Strategy

- Have the the machinery we need to represent functions
 - Treat functions as values
 - Dynamic type checks
 - Substitution for replacing function arguments
 - Already have from Let
- Need to decide:
 - How to substitute within functions/calls

Implementation Strategy

- Have the the machinery we need to represent functions
 - Treat functions as values
 - Dynamic type checks
 - Substitution for replacing function arguments
 - Already have from Let
- Need to decide:
 - How to substitute within functions/calls
 - How to interpret functions/calls

- `{lam {SYMBOL} <expr>}`

- `{lam {SYMBOL} <expr>}`
 - `{lam {x} body}` is anonymous function with argument `x` and body `body`

- `{lam {SYMBOL} <expr>}`
 - `{lam {x} body}` is anonymous function with argument `x` and body `body`
 - `lam` instead of `lambda` to distinguish Curly vs Flit

- `{lam {SYMBOL} <expr>}`
 - `{lam {x} body}` is anonymous function with argument `x` and body `body`
 - `lam` instead of `lambda` to distinguish Curly vs Flit

Syntax and Parsing

- `{lam {SYMBOL} <expr>}`
 - `{lam {x} body}` is anonymous function with argument `x` and body `body`
 - `lam` instead of `lambda` to distinguish Curly vs Flit

```
(define (parse s-exp)
  (cond
    ....
    [(s-exp-match? `{lam {SYMBOL} ANY} s)
     (lamS (s-exp->symbol
              (first (s-exp->list
                      (second (s-exp->list s)))))
            (parse (third (s-exp->list s))))))])
```


- Function application

- Function application
 - Another name for function calls

Parsing Calls

- Function application
 - Another name for function calls
- Same as before, but now we need to allow any expression in function position, not just symbols

Parsing Calls

- Function application
 - Another name for function calls
- Same as before, but now we need to allow any expression in function position, not just symbols
 - Has to be the last case in the parser

- Function application
 - Another name for function calls
- Same as before, but now we need to allow any expression in function position, not just symbols
 - Has to be the last case in the parser
 - Parse as function call if first thing in `{ . . . }` is not a keyword

- Function application
 - Another name for function calls
- Same as before, but now we need to allow any expression in function position, not just symbols
 - Has to be the last case in the parser
 - Parse as function call if first thing in `{ . . . }` is not a keyword

Parsing Calls

- Function application
 - Another name for function calls
- Same as before, but now we need to allow any expression in function position, not just symbols
 - Has to be the last case in the parser
 - Parse as function call if first thing in `{ ... }` is not a keyword

```
[(s-exp-match? `{ANY ANY} s)
 (appS (parse (first (s-exp->list s))
              (parse (second (s-exp->list s)))))]
```

Lambdas in our ASTs

Lambdas in our ASTs

```
(define-type Exp
  ....
  (lamE [arg : Symbol]
        [body : Expr]))
(appE [fun : Exp]
      [arg : Exp])
```

- Similar for ExpS

Functions as Values

- Add a function constructor

Functions as Values

- Add a function constructor

Functions as Values

- Add a function constructor

```
(define-type Value
  (numV [num : Number])
  (boolV [b : Boolean])
  (lamV [arg : Symbol]
        [body : Expr]))
```

- The result of interpretation is called a *value*

Functions as Values

- Add a function constructor

```
(define-type Value
  (numV [num : Number])
  (boolV [b : Boolean])
  (lamV [arg : Symbol]
        [body : Expr]))
```

- The result of interpretation is called a *value*
 - Number, or a function

Functions as Values

- Add a function constructor

```
(define-type Value
  (numV [num : Number])
  (boolV [b : Boolean])
  (lamV [arg : Symbol]
        [body : Expr]))
```

- The result of interpretation is called a *value*
 - Number, or a function
 - Function stores the info we need to call it

Functions as Internalized Expressions

- Notice that `lamE` and `lamV` have the *exact* same fields

Functions as Internalized Expressions

- Notice that `lamE` and `lamV` have the *exact* same fields
 - First-class functions in Curly let us treat *expressions* as values

Functions as Internalized Expressions

- Notice that `lamE` and `lamV` have the *exact* same fields
 - First-class functions in Curly let us treat *expressions* as values
 - A computation to be performed later

Functions as Internalized Expressions

- Notice that `lamE` and `lamV` have the *exact* same fields
 - First-class functions in Curly let us treat *expressions* as values
 - A computation to be performed later
 - In some other context

Functions as Internalized Expressions

- Notice that `lamE` and `lamV` have the *exact* same fields
 - First-class functions in Curly let us treat *expressions* as values
 - A computation to be performed later
 - In some other context
- The value of a function is the code to be run later

Substitution for Calls and Lambda

- appE works just like plus, times, etc.

Substitution for Calls and Lambda

- appE works just like plus, times, etc.
 - Doesn't bind anything, so we just substitute in the parts

Substitution for Calls and Lambda

- `appE` works just like `plus`, `times`, etc.
 - Doesn't bind anything, so we just substitute in the parts
- `lamE` binds its variable

Substitution for Calls and Lambda

- `appE` works just like `plus`, `times`, etc.
 - Doesn't bind anything, so we just substitute in the parts
- `lamE` binds its variable
 - Similar to `let1`

Substitution for Calls and Lambda

- `appE` works just like `plus`, `times`, etc.
 - Doesn't bind anything, so we just substitute in the parts
- `lamE` binds its variable
 - Similar to `let1`
- Don't substitute in a Lambda body if the variable we're replacing matches the function variable

Substitution for Calls and Lambda

- `appE` works just like `plus`, `times`, etc.
 - Doesn't bind anything, so we just substitute in the parts
- `lamE` binds its variable
 - Similar to `let1`
- Don't substitute in a Lambda body if the variable we're replacing matches the function variable
 - Ensures that the function variable shadows any previous declarations

Substitution for Calls and Lambda

- `appE` works just like `plus`, `times`, etc.
 - Doesn't bind anything, so we just substitute in the parts
- `lamE` binds its variable
 - Similar to `let1`
- Don't substitute in a Lambda body if the variable we're replacing matches the function variable
 - Ensures that the function variable shadows any previous declarations

Substitution for Calls and Lambda

- appE works just like plus, times, etc.
 - Doesn't bind anything, so we just substitute in the parts
- lamE binds its variable
 - Similar to let1
- Don't substitute in a Lambda body if the variable we're replacing matches the function variable
 - Ensures that the function variable shadows any previous declarations

```
(define (subst [toReplace : Symbol]
              [replacedBy : Expr]
              [replaceIn : Expr]) : Expr
  (type-case Exp replaceIn
    ....
    [(lamE var body)
     (lamE var
      (if (symbol=? var toReplace)
          body
          (subst toReplace replacedBy body)))]
  ))
```

Evaluating Functions

Evaluating Functions

```
(define (interp expr)
  (type-case Exp interp
    ....
    [(lamE x body)
     (lamV x body)] ))
```

- Nothing to do to turn function into a value

Evaluating Functions

```
(define (interp expr)
  (type-case Exp interp
    ....
    [(lamE x body)
     (lamV x body)] ))
```

- Nothing to do to turn function into a value
 - Just package up the data in the Value type

Evaluating Functions

```
(define (interp expr)
  (type-case Exp interp
    ....
    [(lamE x body)
     (lamV x body)] ))
```

- Nothing to do to turn function into a value
 - Just package up the data in the Value type
 - Store the expression to be evaluated later

Calls

```
(define (interp expr)
  (type-case Exp interp
    [(appE fun arg)
     [(appE fun arg)
      (type-case Value (interp fun)
        [(lamV var body)
         (let* ([argExp (Value->Exp (interp arg))]
                [subbedBody (subst var argExp body)])
           (interp subbedBody))])
      [else
       (error 'interp "Tried to call non-function")]]])])
```

- The thing we're calling might not be a Lambda yet

Calls

```
(define (interp expr)
  (type-case Exp interp
    [(appE fun arg)
     [(appE fun arg)
      (type-case Value (interp fun)
        [(lamV var body)
         (let* ([argExp (Value->Exp (interp arg))]
                [subbedBody (subst var argExp body)])
           (interp subbedBody))])
      [else
       (error 'interp "Tried to call non-function")]]])])
```

- The thing we're calling might not be a Lambda yet
 - So we evaluate it recursively

Calls

```
(define (interp expr)
  (type-case Exp interp
    [(appE fun arg)
     [(appE fun arg)
      (type-case Value (interp fun)
        [(lamV var body)
         (let* ([argExp (Value->Exp (interp arg))]
                [subbedBody (subst var argExp body)])
           (interp subbedBody))])
      [else
       (error 'interp "Tried to call non-function")]]])])
```

- The thing we're calling might not be a Lambda yet
 - So we evaluate it recursively
 - Dynamic check that result is a function

Calls

```
(define (interp expr)
  (type-case Exp interp
    [(appE fun arg)
     [(appE fun arg)
      (type-case Value (interp fun)
        [(lamV var body)
         (let* ([argExp (Value->Exp (interp arg))]
                [subbedBody (subst var argExp body)])
           (interp subbedBody))])
      [else
       (error 'interp "Tried to call non-function")]]])])
```

- The thing we're calling might not be a Lambda yet
 - So we evaluate it recursively
 - Dynamic check that result is a function
- Then, we have a `lamV` and an argument expression

Calls

```
(define (interp expr)
  (type-case Exp interp
    [(appE fun arg)
     [(appE fun arg)
      (type-case Value (interp fun)
        [(lamV var body)
         (let* ([argExp (Value->Exp (interp arg))]
                [subbedBody (subst var argExp body)])
           (interp subbedBody))])
      [else
       (error 'interp "Tried to call non-function")]]])])
```

- The thing we're calling might not be a Lambda yet
 - So we evaluate it recursively
 - Dynamic check that result is a function
- Then, we have a `lamV` and an argument expression
 - Evaluate the argument

Calls

```
(define (interp expr)
  (type-case Exp interp
    [(appE fun arg)
     [(appE fun arg)
      (type-case Value (interp fun)
        [(lamV var body)
         (let* ([argExp (Value->Exp (interp arg))]
                [subbedBody (subst var argExp body)])
           (interp subbedBody)))]
        [else
         (error 'interp "Tried to call non-function")]]])])
```

- The thing we're calling might not be a Lambda yet
 - So we evaluate it recursively
 - Dynamic check that result is a function
- Then, we have a `lamV` and an argument expression
 - Evaluate the argument
 - Replace the function variable with the argument

Calls

```
(define (interp expr)
  (type-case Exp interp
    [(appE fun arg)
     [(appE fun arg)
      (type-case Value (interp fun)
        [(lamV var body)
         (let* ([argExp (Value->Exp (interp arg))]
                [subbedBody (subst var argExp body)])
           (interp subbedBody)))]
        [else
         (error 'interp "Tried to call non-function")]]])])
```

- The thing we're calling might not be a Lambda yet
 - So we evaluate it recursively
 - Dynamic check that result is a function
- Then, we have a `lamV` and an argument expression
 - Evaluate the argument
 - Replace the function variable with the argument
 - Finally, evaluate the substituted body

Example

Example

```
{let1 {double {lam x {* 2 x}}}  
      {+ 3 {double 5}}}}
```

- We'll write in syntax, but `interp` works with AST

Example

```
{let1 {double {lam x {* 2 x}}}  
      {+ 3 {double 5}}}
```

- We'll write in syntax, but interp works with AST
- `{{+ 3 {{lam x {* 2 x}} 5}}}`

Example

```
{let1 {double {lam x {* 2 x}}}  
      {+ 3 {double 5}}}}
```

- We'll write in syntax, but `interp` works with AST
- `{{+ 3 {{lam x {* 2 x}} 5}}}`
 - Substitution from interpreting `let1`

Example

```
{let1 {double {lam x {* 2 x}}}  
      {+ 3 {double 5}}}}
```

- We'll write in syntax, but `interp` works with AST
- `{{+ 3 {{lam x {* 2 x}} 5}}}`
 - Substitution from interpreting `let1`
 - `3` evaluates to itself (e.g. `(numV 3)`)

Example

```
{let1 {double {lam x {* 2 x}}}  
      {+ 3 {double 5}}}}
```

- We'll write in syntax, but `interp` works with AST
- `{{+ 3 {{lam x {* 2 x}} 5}}}`
 - Substitution from interpreting `let1`
 - `3` evaluates to itself (e.g. `(numV 3)`)
- `{+ 3 {{lam x {* 2 x}} 5}}`

Example

```
{let1 {double {lam x {* 2 x}}}  
      {+ 3 {double 5}}}}
```

- We'll write in syntax, but `interp` works with AST
- `{{+ 3 {{lam x {* 2 x}} 5}}}`
 - Substitution from interpreting `let1`
 - `3` evaluates to itself (e.g. `(numV 3)`)
- `{+ 3 {{lam x {* 2 x}} 5}}`
 - Lambda evaluates to `(lamV 'x (timesE ...))`

Example

```
{let1 {double {lam x {* 2 x}}}  
      {+ 3 {double 5}}}}
```

- We'll write in syntax, but `interp` works with AST
- `{{+ 3 {{lam x {* 2 x}} 5}}}`
 - Substitution from interpreting `let1`
 - 3 evaluates to itself (e.g. `(numV 3)`)
- `{+ 3 {{lam x {* 2 x}} 5}}`
 - Lambda evaluates to `(lamV 'x (timesE ...))`
 - 5 evaluates to `numV 5`

Example

```
{let1 {double {lam x {* 2 x}}}  
      {+ 3 {double 5}}}}
```

- We'll write in syntax, but `interp` works with AST
- `{{+ 3 {{lam x {* 2 x}} 5}}}`
 - Substitution from interpreting `let1`
 - 3 evaluates to itself (e.g. `(numV 3)`)
- `{+ 3 {{lam x {* 2 x}} 5}}`
 - Lambda evaluates to `(lamV 'x (timesE ...))`
 - 5 evaluates to `numV 5`
 - Substitution: replace `x` with 5 in `{* 2 x}`

Example

```
{let1 {double {lam x {* 2 x}}}  
      {+ 3 {double 5}}}}
```

- We'll write in syntax, but `interp` works with AST
- `{{+ 3 {{lam x {* 2 x}} 5}}}`
 - Substitution from interpreting `let1`
 - 3 evaluates to itself (e.g. `(numV 3)`)
- `{+ 3 {{lam x {* 2 x}} 5}}`
 - Lambda evaluates to `(lamV 'x (timesE ...))`
 - 5 evaluates to `numV 5`
 - Substitution: replace `x` with 5 in `{* 2 x}`
- `{+ 3 {* 2 5}}`

Example

```
{let1 {double {lam x {* 2 x}}}  
      {+ 3 {double 5}}}}
```

- We'll write in syntax, but `interp` works with AST
- `{{+ 3 {{lam x {* 2 x}} 5}}}`
 - Substitution from interpreting `let1`
 - 3 evaluates to itself (e.g. `(numV 3)`)
- `{+ 3 {{lam x {* 2 x}} 5}}`
 - Lambda evaluates to `(lamV 'x (timesE ...))`
 - 5 evaluates to `numV 5`
 - Substitution: replace `x` with 5 in `{* 2 x}`
- `{+ 3 {* 2 5}}`
 - There was computation left to do in the function body

Example

```
{let1 {double {lam x {* 2 x}}}  
      {+ 3 {double 5}}}}
```

- We'll write in syntax, but `interp` works with AST
- `{{+ 3 {{lam x {* 2 x}} 5}}}`
 - Substitution from interpreting `let1`
 - 3 evaluates to itself (e.g. `(numV 3)`)
- `{+ 3 {{lam x {* 2 x}} 5}}`
 - Lambda evaluates to `(lamV 'x (timesE ...))`
 - 5 evaluates to `numV 5`
 - Substitution: replace `x` with 5 in `{* 2 x}`
- `{+ 3 {* 2 5}}`
 - There was computation left to do in the function body
 - Why `interp` the body when interpreting a call

Example

```
{let1 {double {lam x {* 2 x}}}  
      {+ 3 {double 5}}}}
```

- We'll write in syntax, but `interp` works with AST
- `{{+ 3 {{lam x {* 2 x}} 5}}}`
 - Substitution from interpreting `let1`
 - 3 evaluates to itself (e.g. `(numV 3)`)
- `{+ 3 {{lam x {* 2 x}} 5}}`
 - Lambda evaluates to `(lamV 'x (timesE ...))`
 - 5 evaluates to `numV 5`
 - Substitution: replace `x` with 5 in `{* 2 x}`
- `{+ 3 {* 2 5}}`
 - There was computation left to do in the function body
 - Why `interp` the body when interpreting a call
 - Get `{+ 3 10}`, finally `(numV 30)`

Capturing the environment

- Functions might contain *free variables*

Capturing the environment

- Functions might contain *free variables*
 - Variables that are not bound/defined by the function itself

Capturing the environment

- Functions might contain *free variables*
 - Variables that are not bound/defined by the function itself
- Subst *will* replace those variables when concrete values are given

Capturing the environment

- Functions might contain *free variables*
 - Variables that are not bound/defined by the function itself
- Subst *will* replace those variables when concrete values are given

Capturing the environment

- Functions might contain *free variables*
 - Variables that are not bound/defined by the function itself
- Subst *will* replace those variables when concrete values are given

```
{letvar  
  f {lam {x} {lam {y} {+ x y}}}  
  {f 3}}
```

- For any x , f produces another function that adds its argument to x

Capturing the environment

- Functions might contain *free variables*
 - Variables that are not bound/defined by the function itself
- Subst *will* replace those variables when concrete values are given

```
{letvar  
  f {lam {x} {lam {y} {+ x y}}}  
  {f 3}}
```

- For any x , f produces another function that adds its argument to x
- x is *free* in $\{\text{lam } \{y\} \{+ x y\}\}$

Capturing the environment

- Functions might contain *free variables*
 - Variables that are not bound/defined by the function itself
- Subst *will* replace those variables when concrete values are given

```
{letvar  
  f {lam {x} {lam {y} {+ x y}}}  
  {f 3}}
```

- For any x , f produces another function that adds its argument to x
- x is *free* in $\{\text{lam } \{y\} \{+ x y\}\}$
- Interp replaces f with the fun

Capturing the environment

- Functions might contain *free variables*
 - Variables that are not bound/defined by the function itself
- Subst *will* replace those variables when concrete values are given

```
{letvar  
  f {lam {x} {lam {y} {+ x y}}}  
  {f 3}}
```

- For any x , f produces another function that adds its argument to x
- x is *free* in $\{\text{lam } \{y\} \{+ x y\}\}$
- Interp replaces f with the fun
- Calling f with 3 replaces x with 3

Capturing the environment

- Functions might contain *free variables*
 - Variables that are not bound/defined by the function itself
- Subst *will* replace those variables when concrete values are given

```
{letvar  
  f {lam {x} {lam {y} {+ x y}}}  
  {f 3}}
```

- For any x , f produces another function that adds its argument to x
- x is *free* in $\{\text{lam } \{y\} \{+ x y\}\}$
- Interp replaces f with the fun
- Calling f with 3 replaces x with 3
 - Result: $\{\text{lam } \{y\} \{+ 3 y\}\}$

Capturing the environment

- Functions might contain *free variables*
 - Variables that are not bound/defined by the function itself
- Subst *will* replace those variables when concrete values are given

```
{letvar  
  f {lam {x} {lam {y} {+ x y}}}  
  {f 3}}
```

- For any x , f produces another function that adds its argument to x
- x is *free* in $\{\text{lam } \{y\} \{+ x y\}\}$
- Interp replaces f with the fun
- Calling f with 3 replaces x with 3
 - Result: $\{\text{lam } \{y\} \{+ 3 y\}\}$
 - e.g. The function that adds 3 to its argument

Capturing the environment

- Functions might contain *free variables*
 - Variables that are not bound/defined by the function itself
- Subst *will* replace those variables when concrete values are given

```
{letvar  
  f {lam {x} {lam {y} {+ x y}}}  
  {f 3}}
```

- For any x , f produces another function that adds its argument to x
- x is *free* in $\{\text{lam } \{y\} \{+ x y\}\}$
- Interp replaces f with the fun
- Calling f with 3 replaces x with 3
 - Result: $\{\text{lam } \{y\} \{+ 3 y\}\}$
 - e.g. The function that adds 3 to its argument
- Substitution lets us build new functions at run time based on values to other functions

Our Language Is Now Turing Complete!

- Curly-Lambda can simulate every computer program ever written

Our Language Is Now Turing Complete!

- Curly-Lambda can simulate every computer program ever written
 - Not counting syscalls, IO, networking, effects, etc.

Our Language Is Now Turing Complete!

- Curly-Lambda can simulate every computer program ever written
 - Not counting syscalls, IO, networking, effects, etc.
- Just because we *can* write an equivalent Curly-Lambda program, doesn't mean it's easy/succinct

Our Language Is Now Turing Complete!

- Curly-Lambda can simulate every computer program ever written
 - Not counting syscalls, IO, networking, effects, etc.
- Just because we *can* write an equivalent Curly-Lambda program, doesn't mean it's easy/succinct
- We can now write Curly-Lambda programs that run forever

Our Language Is Now Turing Complete!

- Curly-Lambda can simulate every computer program ever written
 - Not counting syscalls, IO, networking, effects, etc.
- Just because we *can* write an equivalent Curly-Lambda program, doesn't mean it's easy/succinct
- We can now write Curly-Lambda programs that run forever

Our Language Is Now Turing Complete!

- Curly-Lambda can simulate every computer program ever written
 - Not counting syscalls, IO, networking, effects, etc.
- Just because we *can* write an equivalent Curly-Lambda program, doesn't mean it's easy/succinct
- We can now write Curly-Lambda programs that run forever

```
{let1 f {lam x {x x}}  
  {f f}}
```

- Replaces `f` with `{{lam {x} {x x}} {lam x {x x}}}` calling itself

Our Language Is Now Turing Complete!

- Curly-Lambda can simulate every computer program ever written
 - Not counting syscalls, IO, networking, effects, etc.
- Just because we *can* write an equivalent Curly-Lambda program, doesn't mean it's easy/succinct
- We can now write Curly-Lambda programs that run forever

```
{let1 f {lam x {x x}}  
  {f f}}
```

- Replaces `f` with `{{lam {x} {x x}} {lam x {x x}}}` calling itself
- Function call: takes body `{x x}`, replaces `x` with argument `{lam {x} {x x}}`

Our Language Is Now Turing Complete!

- Curly-Lambda can simulate every computer program ever written
 - Not counting syscalls, IO, networking, effects, etc.
- Just because we *can* write an equivalent Curly-Lambda program, doesn't mean it's easy/succinct
- We can now write Curly-Lambda programs that run forever

```
{let1 f {lam x {x x}}  
  {f f}}
```

- Replaces `f` with `{{lam {x} {x x}} {lam x {x x}}}` calling itself
- Function call: takes body `{x x}`, replaces `x` with argument `{lam {x} {x x}}`
- Result is `{{lam {x} {x x}} {lam {x} {x x}}}`, exactly what we started with!

Our Language Is Now Turing Complete!

- Curly-Lambda can simulate every computer program ever written
 - Not counting syscalls, IO, networking, effects, etc.
- Just because we *can* write an equivalent Curly-Lambda program, doesn't mean it's easy/succinct
- We can now write Curly-Lambda programs that run forever

```
{let1 f {lam x {x x}}  
  {f f}}
```

- Replaces `f` with `{{lam {x} {x x}} {lam x {x x}}}` calling itself
- Function call: takes body `{x x}`, replaces `x` with argument `{lam {x} {x x}}`
- Result is `{{lam {x} {x x}} {lam {x} {x x}}}`, exactly what we started with!
- `interp` runs forever

A First Taste of Laziness

- How we implement `if` really matters now

A First Taste of Laziness

- How we implement `if` really matters now
 - Code might run forever

A First Taste of Laziness

- How we implement `if` really matters now
 - Code might run forever
 - Don't want to run code in the branch we don't take

A First Taste of Laziness

- How we implement `if` really matters now
 - Code might run forever
 - Don't want to run code in the branch we don't take
- What if we did this?

A First Taste of Laziness

- How we implement `if` really matters now
 - Code might run forever
 - Don't want to run code in the branch we don't take
- What if we did this?

A First Taste of Laziness

- How we implement `if` really matters now
 - Code might run forever
 - Don't want to run code in the branch we don't take
- What if we did this?

```
;; BAD! Don't do this
(define (interp expr
  (type-case Exp expr
    [(cndE test thn els)
      (let ([thenVal (interp thn)]
            [elseVal (interp els)])
        (if (= 0 (checkAndGetNum (interp test)))
            thenVal
            elseVal)))]))
(run ~{if {zero? 0}
          {+ 1 1}
          {let1 f {lam x {x x}} {f f}}})
```

- Loops because it evaluates the untaken branch